# Mars Habitat Automation Platform

## Distributed Event-Driven System for Habitat Monitoring & Control

Deda Daniel • Macrì Francesco • Pagliarini Paolo • Torresi Flavio

Laboratory of Advanced Programming – Hackathon Exam 2025/2026

# Agenda


01 **Project Overview**
Problem statement and objectives

02 **System Architecture**
Microservices and event-driven design

03 **Data Ingestion**
REST polling and SSE streaming

04 **Automation Engine**
IF-THEN rule processing

05 **Frontend Application**
Real-time dashboard features

06 **Technology Stack**
Tools and frameworks used

07 **Conclusions**
Achievements and future work

# The Challenge

A Mars habitat's automation stack has partially failed, leaving devices operating with incompatible protocols.

### Multiple Data Sources

REST sensors, SSE streams, WebSocket telemetry

### Heterogeneous Formats

Different payload schemas requiring normalization

### Real-time Requirements

Immediate response to environmental changes

### Automation Needs

Rules-based actuator control for habitat safety

# Our Solution

A distributed event-driven platform with unified monitoring and control

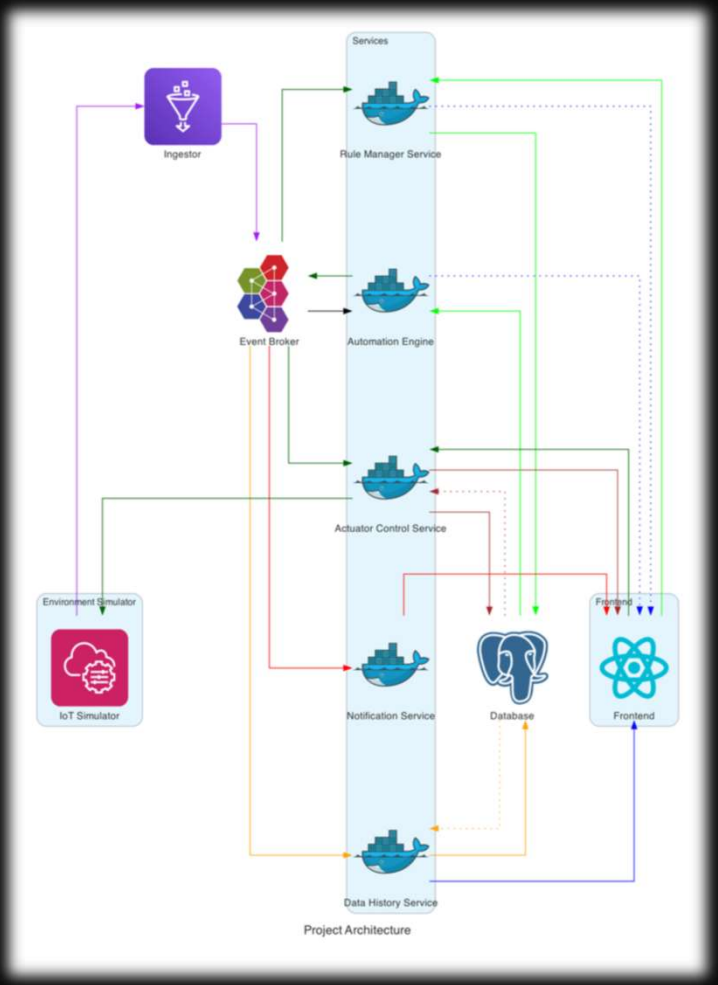
**Unified Ingestion**
REST polling + SSE streams normalized to single event schema

**Event-Driven Core**
RabbitMQ message broker for decoupled microservices

**Smart Automation**
IF-THEN rules with automatic actuator control

**Real-Time Dashboard**
React SPA with live updates and manual controls

# System Architecture



# Microservices Architecture

### Ingestion
8001

REST polling & SSE streams
Schema normalization

### Automation Engine
8002

Rule evaluation
Event-driven triggers

### Rule Manager
8003

CRUD API for automation rules persistence

### Notification
8004

Real-time SSE alerts
Event broadcasting

### Actuator Control
8005

Device command execution
Audit logging

### Data History
8006

Historical queries
In-memory cache

# Infrastructure Services

## RabbitMQ Message Broker

- AMQP protocol (port 5672)
- Topic exchange: mars_events
- Event-driven pub/sub architecture
- Durable queues with TTL
- Management UI on port 15672

## PostgreSQL Database

- PostgreSQL 16 (port 5433)
- automation_rules table
- sensor_readings history
- actuator_commands audit
- Persistent volume storage

## Unified Event Schema

{ event_id, sensor_id, timestamp, metric, value, unit, source, status, raw_schema }

All sensor data normalized to a single format • REST polls + SSE streams → unified events

# Data Ingestion Service

## REST Sensors (Polling)

- greenhouse_temperature
- entrance_humidity
- co2_hall
- hydroponic_ph
- water_tank_level
- corridor_pressure
- air_quality_pm25
- air_quality_voc

## Telemetry Streams (SSE)

- solar_array
- radiation
- life_support
- thermal_loop
- power_bus
- power_consumption
- airlock

### Normalization Pipeline

8 different REST schemas + 7 telemetry schemas → Single unified event format
Published to RabbitMQ with routing key: events.sensor.{sensor_id}

# Raw Data to JSON (Normalization)

## Standardizing Telemetry and Events

### Raw Telemetry (Varies)

```
{
  "sensor": "DHT22",
  "data": {
    "temp": 39.99,
    "flow": 12.0
  }
}
```

### Unified Event JSON

```
{
  "event_id": "evt_101",
  "sensor_id": "thermal_loop",
  "timestamp": "2026-03-
10T12:00:00Z",
  "metric": "temperature_c",
  "value": 39.99,
  "unit": "C",
  "source": "sse",
  "status": "warning"
}
```

# In-Memory Cache & Mapping

Ingestion service maintains the latest values for every sensor, exposed via GET /sensors/latest to the frontend with real-time updates.

## Event Mapping Strategies

| | | |
|---|---|---|
| `rest.scalar.v1` | → | 1 direct event (1:1 mapping) |
| `rest.chemistry.v1` | → | 1 event per measurement in the measurements array |
| `rest.particulate.v1` | → | 3 separated events (PM1, PM2.5, PM10) |
| `rest.level.v1` | → | 2 events (level_pct in % and level_liters in L) |
| `topic.power.v1/environment.v1` | → | 1 event per metric |
| `topic.thermal_loop.v1` | → | 2 events (temperature_c, flow_l_min) |
| `topic.airlock.v1` | → | 1 event (last_state encoded as status) |

# Automation Engine

## Event-Driven Rule Processing

1. **Receive** — RabbitMQ event
2. **Parse** — Extract sensor_id
3. **Match** — Find rules
4. **Evaluate** — Check condition
5. **Execute** — Publish command

### IF-THEN Rule Model

IF <sensor_id> <operator> <value> THEN set <actuator_id> to ON | OFF

Operators: < <= = > >= • Example: IF greenhouse_temperature > 28 THEN set cooling_fan to ON

# Frontend Application

## Technology Stack

- React 18 + TypeScript
- Vite 6 Build Tool
- React Router 7
- Tailwind CSS 4
- Recharts for Charts
- Radix UI Components

## Application Pages

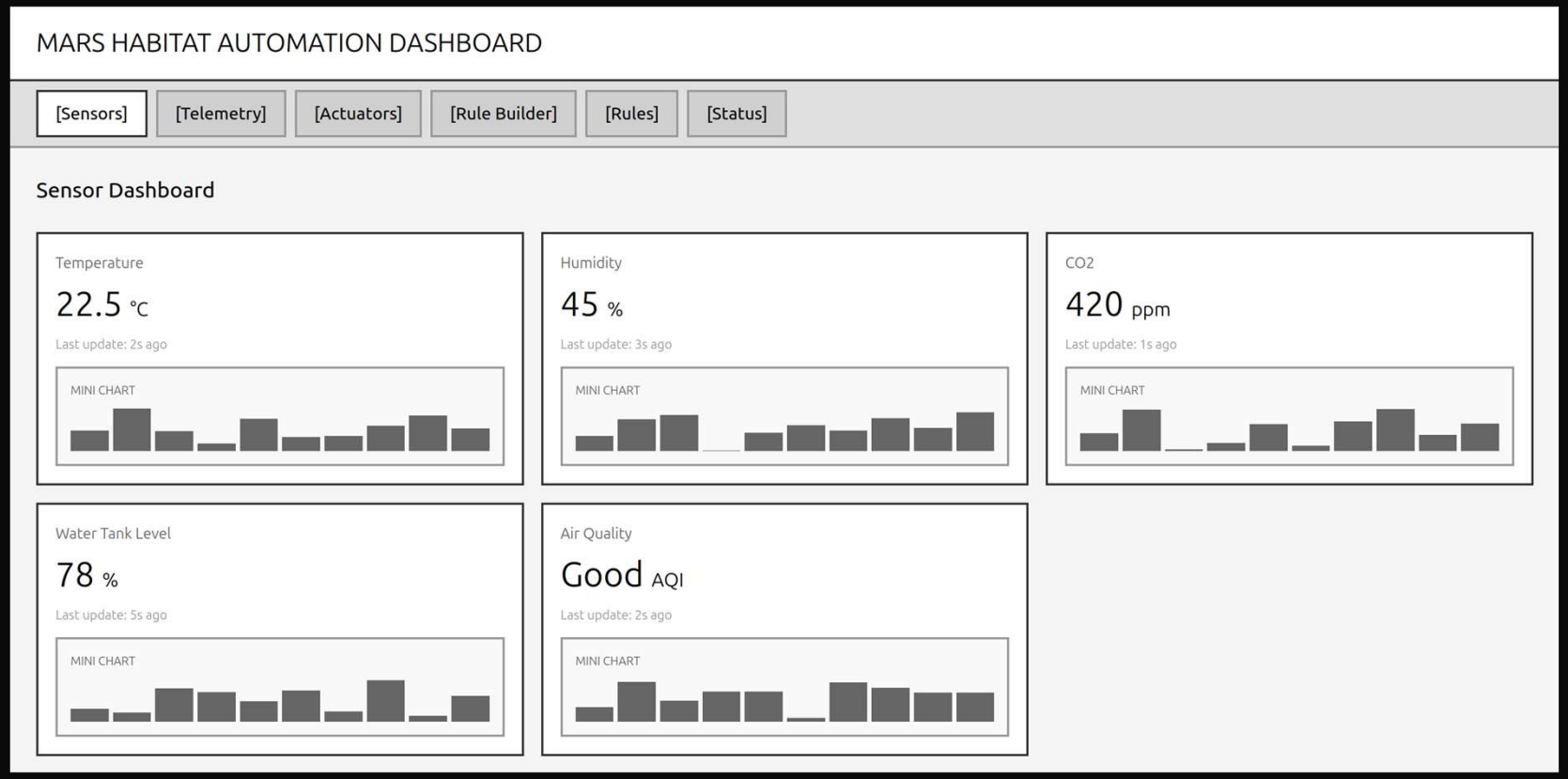
| | |
|---|---|
| **Sensors** | REST sensors dashboard with real-time updates |
| **Telemetry** | Streaming data visualization with charts |
| **Actuators** | Device control and command history |
| **Rule Builder** | Create and edit automation rules |
| **Rules** | List, enable/disable, delete rules |
| **Status** | System health and notification log |

## Real-Time Features

Server-Sent Events (SSE) for live notifications • Aggressive polling for sensor updates
React state management for instant UI feedback • Recharts for live data visualization

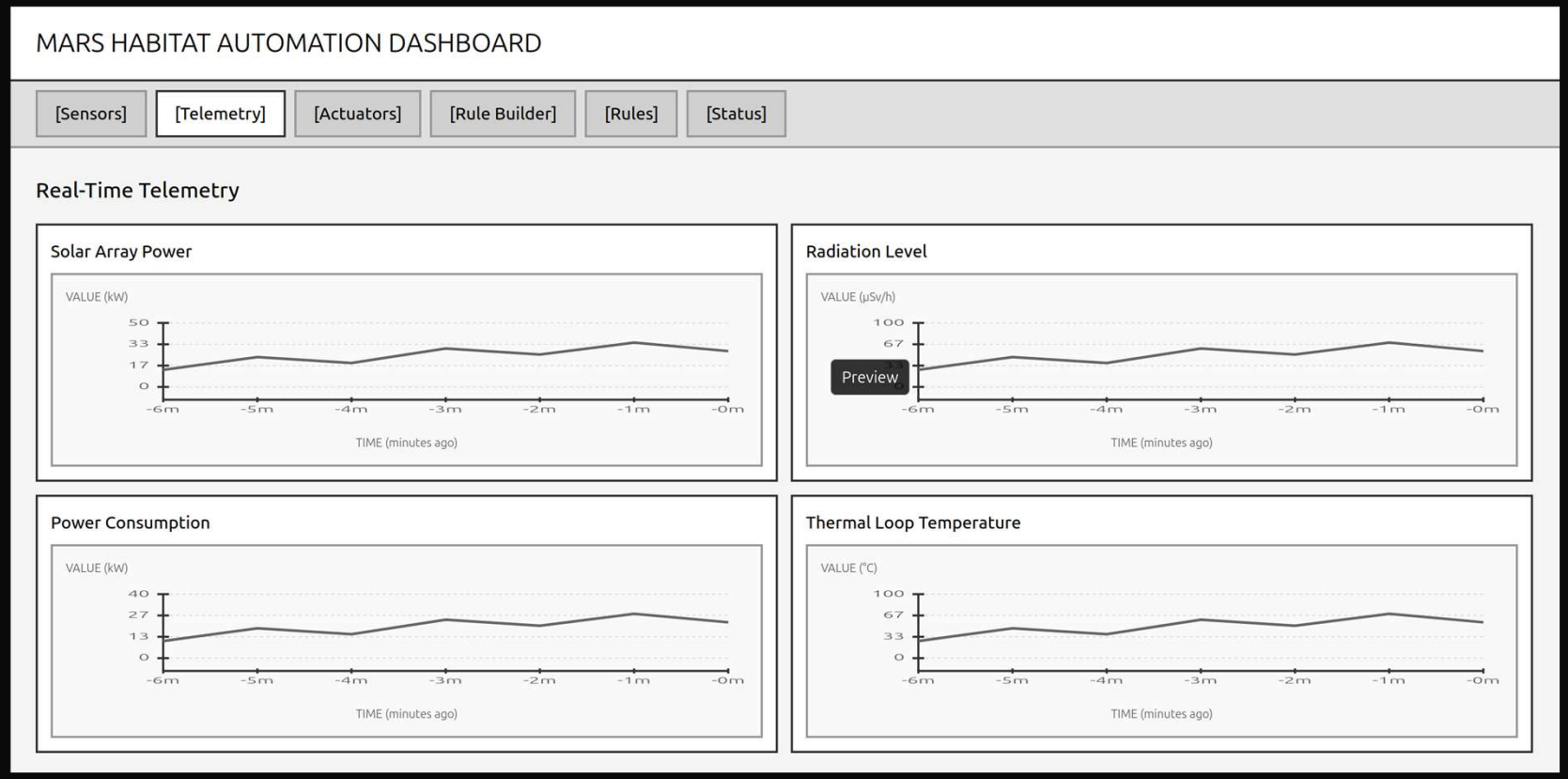
# Sensors Dashboard

Real-time REST sensor monitoring with status indicators



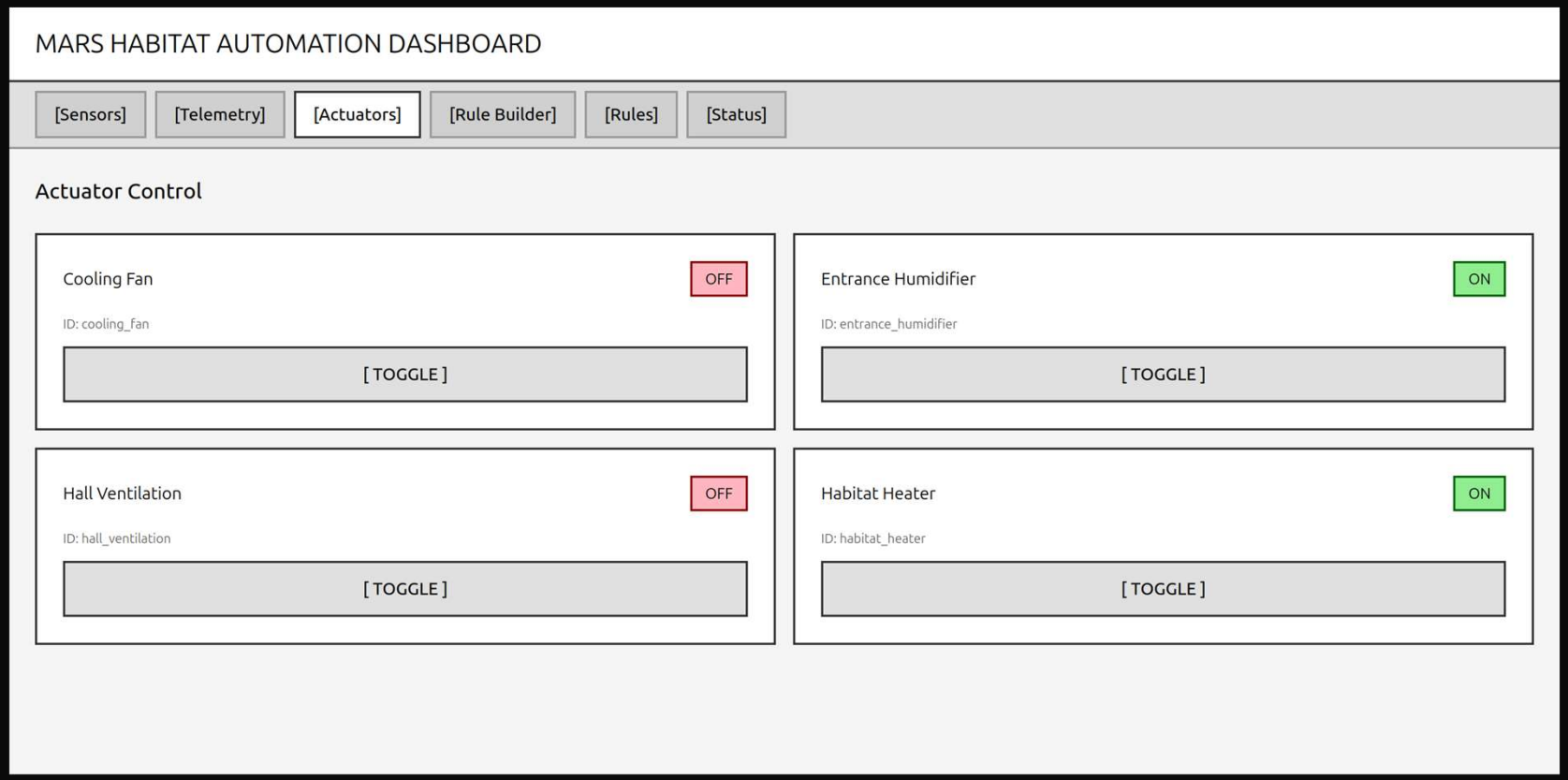
# Telemetry Streaming

SSE-powered live data visualization



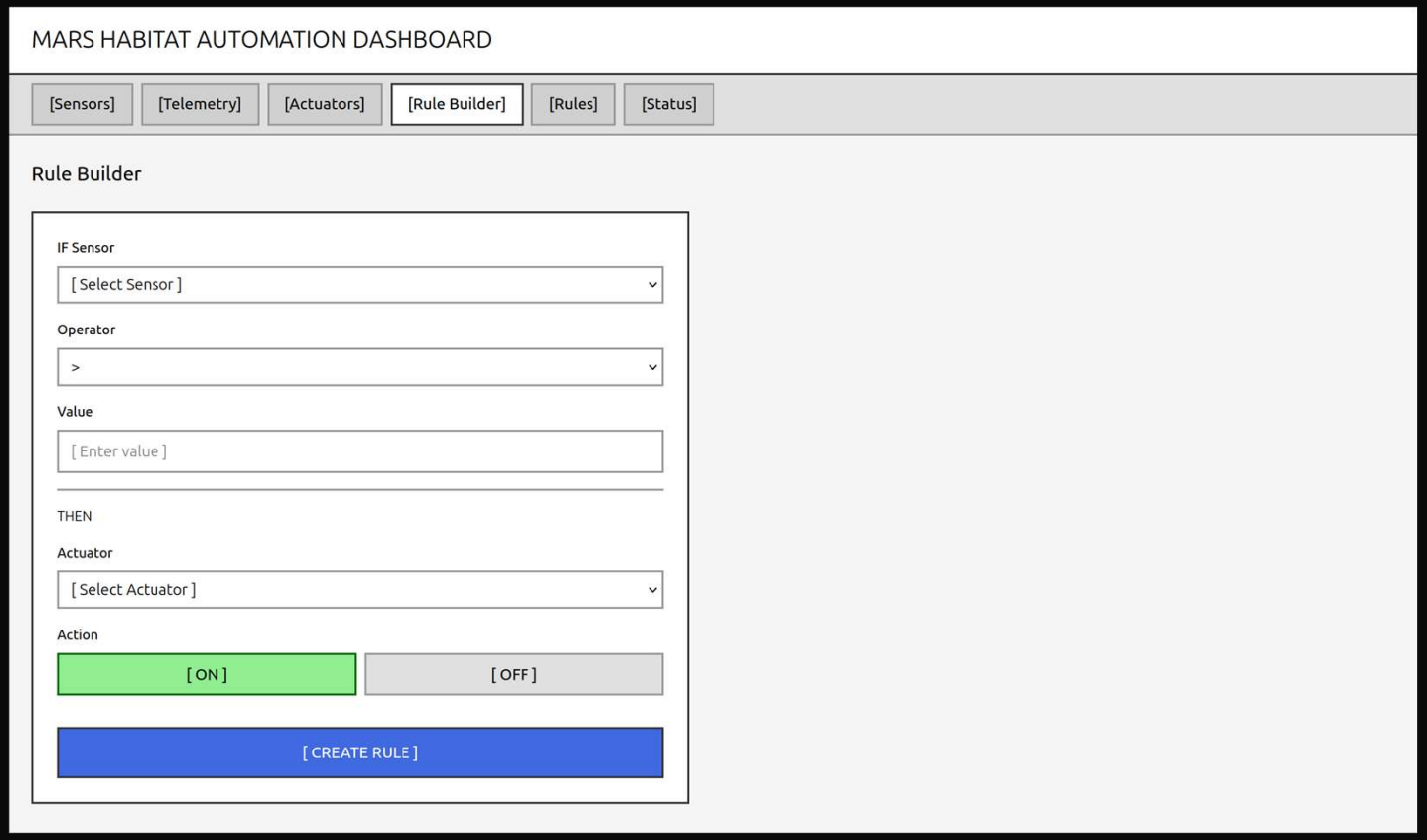
# Actuator Control

Manual control and command history audit trail



# Rule Builder

Create IF-THEN automation rules visually



# Rules Management

List, toggle, edit, and delete automation rules

MARS HABITAT AUTOMATION DASHBOARD

[Sensors] [Telemetry] [Actuators] [Rule Builder] [Rules] [Status]

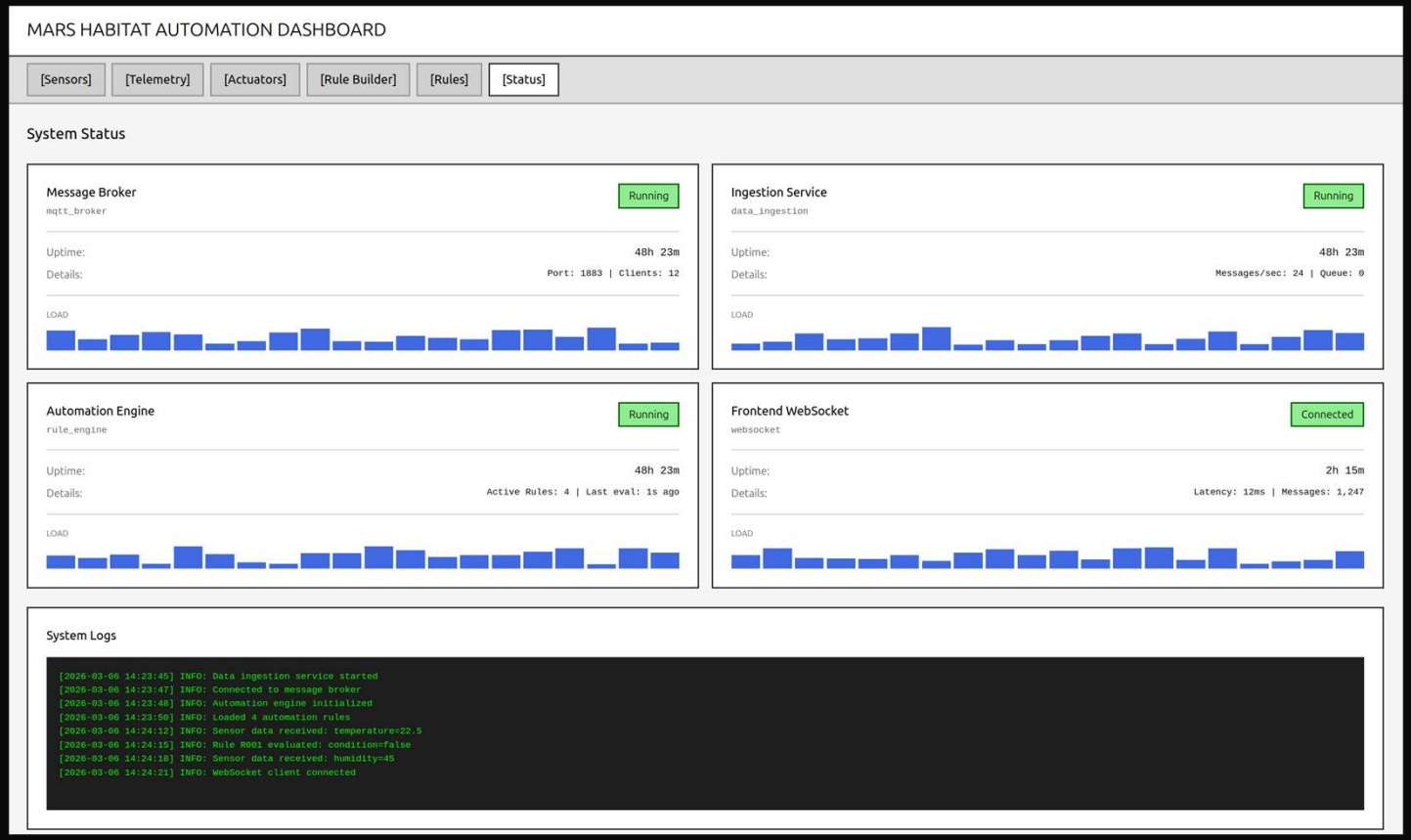
## Active Automation Rules

| Rule ID | Condition | Action | Status | Last Triggered |
|---|---|---|---|---|
| R001 | temperature > 25 | cooling_fan = ON | Active | 2 min ago |
| R002 | humidity < 40 | entrance_humidifier = ON | Active | 15 min ago |
| R003 | co2 > 600 | hall_ventilation = ON | Active | Never |
| R004 | temperature < 18 | habitat_heater = ON | Inactive | 3 hours ago |

[ EDIT ] [ DELETE ] [ ENABLE/DISABLE ]

# System Status

Health monitoring and notification center



# Database Design

### automation_rules

- id (SERIAL PK)
- name, description
- sensor_id, operator
- threshold_value, threshold_unit
- actuator_id, actuator_action
- is_active (BOOLEAN)
- created_at, updated_at

### sensor_readings

- id (BIGSERIAL PK)
- sensor_id (VARCHAR)
- value (DECIMAL)
- unit (VARCHAR)
- source (rest/stream)
- recorded_at (TIMESTAMP)
- created_at (TIMESTAMP)

### actuator_commands

- id (BIGSERIAL PK)
- actuator_id (VARCHAR)
- previous_state, new_state
- source (auto/manual)
- reason (TEXT)
- rule_id (FK)
- executed_at (TIMESTAMP)

# Data Flow Architecture

## Event Routing Keys

**Sensor Events:**
events.sensor.{sensor_id}

**Actuator Commands:**
commands.actuator.{actuator_id}

**Notifications:**
events.#

# Technology Stack

## Backend Services

- Python 3.11+
- FastAPI (Async)
- aio-pika (RabbitMQ)
- asyncpg + SQLAlchemy
- httpx (HTTP Client)
- Pydantic (Validation)
- Uvicorn (ASGI)

## Frontend

- React 18
- TypeScript
- Vite 6
- React Router 7
- Tailwind CSS 4
- Recharts
- Radix UI

## Infrastructure

- PostgreSQL 16
- RabbitMQ (AMQP)
- Docker
- Docker Compose
- Nginx (Reverse Proxy)

Containerized Microservices with Event-Driven Architecture

# Key User Stories

## Monitoring

- Real-time sensor values
- Live telemetry from SSE
- Automatic dashboard updates
- Status indicators (ok/warning)
- Sensor filtering by category

## Control

- View actuator ON/OFF states
- Manual actuator toggle
- Automatic rule-based control
- Command history audit

## Automation

- Create IF-THEN rules
- View all automation rules
- Edit existing rules
- Delete/enable/disable
- Real-time trigger alerts

## Analytics

- Live time charts
- Historical data queries
- System health summary
- Connectivity monitoring

# Conclusions

## Project Achievements

- Distributed microservices architecture with event-driven communication
- Real-time monitoring and control of simulated Mars habitat
- IF-THEN automation rules with automatic actuator control
- Modern React dashboard with SSE-based live updates
- Containerized deployment with Docker Compose
- Unified event schema for heterogeneous data sources

## Future Enhancements

→ **Machine Learning:** Predictive maintenance and anomaly detection
→ **Mobile App:** Remote monitoring and push notifications
→ **Advanced Alerting:** Email/SMS notifications for critical events

# Thank You

Deda Daniel • Macrì Francesco • Pagliarini Paolo • Torresi Flavio

Laboratory of Advanced Programming – Hackathon Exam 2025/2026